

AI 博客平台 CI/CD 实现流程

基于仓库实现整理 · workflow: `.github/workflows/deploy.yml` · 2026-09-23

一、总览

整条流水线一句话：**push 到 main 触发 GitHub Actions，按改动路径只构建受影响端的 Docker 镜像，推到腾讯云 TCR 镜像仓库，再 SSH 到服务器执行 `compose pull + up` 完成发版。**服务器（2C4G）只做拉取和重启，不参与构建。

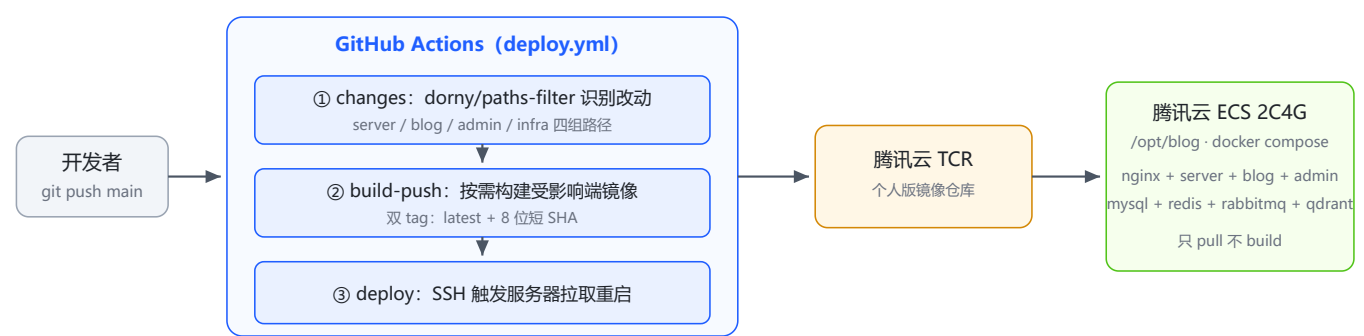


图 1 · push main 到生产发版的完整链路

三个关键设计决策

决策	理由
服务器只 pull 不 build	ECS 是 2C4G 低配实例，Go/Node 构建又吃内存又吃 CPU，构建放在 GitHub Actions 的境外 runner 上，服务器零构建压力。
路径过滤，只建受影响端	Monorepo 三端 + 编排层，改 admin 不必重建 server。四组过滤规则把每次发版的构建时间压到最低（通常只建一个镜像）。
镜像双 tag (latest + 短 SHA)	<code>latest</code> 供日常发版， <code>SHA tag</code> 是不可变的历史版本锚点——回滚时把服务器 <code>.env</code> 里的 tag 换回旧 SHA 即可，不需要重新构建。

二、触发与改动识别 (changes job)

2.1 触发方式

触发	行为
push 到 main	自动执行完整流水线（识别 → 构建 → 部署）。
workflow_dispatch 手动触发	可勾选 <code>force_all</code> ：跳过路径过滤，三个镜像全部重建。用于首次部署、或需要全量重推的场景。
concurrency: deploy	同一时间只允许一条流水线， <code>cancel-in-progress: false</code> —— 上一次没跑完时新的排队等待而不是取消，避免服务器上两次 <code>pull/up</code> 互相踩踏。

2.2 路径过滤规则

用 `dorny/paths-filter@v3` 对本次推送的提交做 diff 匹配，输出四个布尔值：

过滤组	匹配路径	影响
server	server/**、docker/Dockerfile.server	重建 blog-server 镜像
blog	web/blog/**、docker/Dockerfile.blog	重建 blog-blog 镜像
admin	web/admin/**、 docker/Dockerfile.admin	重建 blog-admin 镜像
infra	docker/compose.prod.yml、 docker/nginx/**	不建镜像，但触发部署阶段（让服务器重启 nginx 重读配置）

checkout 时显式 `fetch-depth: 0`（完整历史）。一次推多个提交时，浅克隆会让 `paths-filter` 的 diff 范围算错——只看到最后一个提交的改动。

三、构建并推送镜像 (build-push job)

3.1 版本 tag

```
short=${GITHUB_SHA::8} # 取当前 commit SHA 前 8 位
```

每个镜像同时打两个 tag 推送：`{REGISTRY}/{NAMESPACE}/blog-server:latest` 与 `blog-server:9e017f0a`（示例）。

3.2 登录与推送

用 `docker/login-action@v3` 登录 TCR（账号 ID + 预设密码，来自仓库 Secrets），随后 `docker/build-push-action@v6` 构建 + 直接推送。三个镜像的构建步骤各自带 `if` 条件：只有 `paths-filter` 判定受影响（或 `force_all`）才执行；三端都没变时整个 job 跳过。

Runner 在境外，所以构建参数显式覆盖 Dockerfile 里默认的国内镜像源：server 用 `GOPROXY=https://proxy.golang.org,direct`，blog/admin 用 `NPM_REGISTRY=https://registry.npmjs.org`。本地手工构建则走默认的 `goproxy.cn` / `npmmirror`。

3.3 三个生产镜像的构建要点

镜像	构建方式	要点
blog-server	golang:1.25-alpine 多阶段 → alpine:3.20 运行层	CGO_ENABLED=0 静态编译，产物不依赖 glibc；先拷 go.mod/go.sum 单独 go mod download 命中缓存；运行层装 tzdata（东八区日志）、ca-certificates（出网调 LLM）、git（任务流水线克隆仓库必需）；config.yaml 不打入镜像，运行时只读挂载。
blog-blog	node:22-alpine 多阶段，Next.js output: standalone	API_BASE=http://server:8080 在构建期固化（Next rewrites 是构建期写死的），指向 compose 网络内的 server 服务名；运行层只拷 standalone 产物 + static 资源，node server.js 启动。
blog-admin	node:22-alpine 构建 → nginx:1.27-alpine 托管	npm run build 出 dist 后由 nginx 托管；容器内置一份 nginx conf: SPA 路由回退 try_files \$uri /index.html、带内容哈希的 /assets/ 永久缓存、/api/ 反代到 server:8080。

四、服务器拉取并重启（deploy job）

4.1 触发条件（这段 if 值得读）

```
if: always() && needs.build-push.result != 'failure'
    && (needs.build-push.result == 'success' || needs.changes.outputs.infra == 'true')
```

- 构建失败 → 永不发版（坏镜像不能上服务器）；
- 构建成功（至少建了一个镜像）→ 正常发版；
- 构建被跳过但只有 infra 变更（改了 compose / nginx conf）→ 也要发版：拉不到新镜像无所谓，up -d 会应用新编排、restart nginx 重读配置。

4.2 部署脚本

用 appleboy/ssh-action@v1.2.2 以部署专用私钥 SSH 到服务器执行：

```
set -e                                # 任一命令失败即中止
cd /opt/blog
docker compose -f compose.prod.yml pull server blog admin  # 只拉三端自有镜像
docker compose -f compose.prod.yml up -d                  # 应用新镜像与编排变更
docker compose -f compose.prod.yml restart nginx          # bind-mount 的 conf 变更需重启重读
docker image prune -f                                     # 清掉被顶替的旧镜像层，防磁盘吃满
```

两个坑都真实踩过：① action 的 script_stop: true 实测未生效，服务器脚本必须自己 set -e，否则中途失败也报绿（2026-09-22 首次部署假绿教训）；② nginx conf 是 bind-mount 单文件，up -d 感知不到内容变化，必须显式 restart nginx。

中间件镜像 (mysql/redis/rabbitmq/qdrant) 不在 pull 列表里——它们走腾讯云内网镜像加速，版本固定，不属于每次发版范围。

五、服务器运行时编排 (compose.prod.yml)

服务器 /opt/blog 下一份 compose 起 **8 个服务**：统一入口 nginx + 三端 + 四中间件。

5.1 服务与启动顺序

服务	健康检查 / 依赖
mysql 8.0	mysqladmin ping ; utf8mb4 字符集；数据落 named volume。
redis 7-alpine	redis-cli ping ; --requirepass 密码启动。
rabbitmq 3-management	rabbitmq-diagnostics ping 。
qdrant 1.19.1	/dev/tcp 探活（镜像内无 curl/wget）；版本 钉死 ，防升级改向量格式破坏本地索引兼容。
server (Go)	depends_on 四个中间件全部 service_healthy 才启动；自身用 /metrics 探活（无鉴权、不碰业务库），start_period 15s 。
blog (Next.js)	depends_on server healthy (RSC 服务端取数依赖后端) 。
admin (静态)	纯 nginx 托管，无探活。
nginx 1.27-alpine	统一入口，最后启动；conf 三件套 bind-mount 只读挂载。

5.2 安全与运维约束

- 中间件端口一律不映射公网，对外只有 nginx 的 80/443（备案期临时开放 8081/8082 预览口）；
- RabbitMQ 管理界面只绑 127.0.0.1:15672，公网不可达，需要看时走 SSH 隧道；
- nginx 兜底块对 IP 直连返回 **444**（不配域名访问不到任何服务）；
- 全部服务 restart: unless-stopped；日志统一限额 json-file 10m × 3 个，防日志吃满磁盘；
- 四中间件数据全部落 named volume，Qdrant 向量库数据必须持久化。

5.3 配置注入的三层分工

层	内容
config.yaml（只读挂载）	唯一的完整配置文件， 不含任何密钥 ；随代码进 git、随镜像外挂载进容器。
environment（compose 内）	拓扑固定项：覆盖 config 里的本机开发地址——MYSQL_HOST=mysql、REDIS_ADDR=redis:6379、QDRANT_BASE_URL=http://qdrant:6333、SERVER_MODE=release 。
env_file: .env（不入 git）	密钥与主机相关项：数据库/Redis/RabbitMQ 密码、LLM API Key、三端镜像地址（SERVER_IMAGE 等指向 TCR）。

六、流量入口 (nginx)

请求	去向
1jl0628.cn / <code>www</code> 的 /	反代 <code>blog:3000</code> (Next.js SSR/ISR/静态资源), 保留 Upgrade 头。
同域 <code>/api/</code>	反代 <code>server:8080</code> (文章列表/详情、SVG 配图、点赞、登录等)。
<code>/api/v1/portal/ask</code> (RAG 问答 SSE)	单独 location: 关闭缓冲 (<code>proxy_buffering off</code>) + 读超时拉到 300s, 否则前端收不到逐字流式输出。
admin 子域整站	转给 admin 容器 (其内置 nginx 再把 <code>/api/</code> 反代回 <code>server:8080</code>)。
IP 直连 (无域名 Host)	兜底块返回 444。

七、回滚

- 在 TCR 控制台 (或 `docker pull` 验证) 确认要回退到的短 SHA tag 存在——每次发版都留有;
- 改服务器 `/opt/blog/.env` 中对应镜像行, 如 `SERVER_IMAGE=.../blog-server:70f0982a`;
- `docker compose -f compose.prod.yml up -d` 完成回退, 随后可手工 `docker image prune -f`。

八、一次性前置配置

8.1 GitHub 仓库 (Settings → Secrets and variables → Actions)

类型	名称	说明
Variables	<code>REGISTRY</code>	TCR 地址, 如 <code>registry.cn-guangzhou.tencentcloudcr.com</code> (与 ECS 同地域, 拉取走内网)。
	<code>NAMESPACE</code>	TCR 个人版命名空间。
Secrets	<code>TCR_USERNAME</code>	腾讯云账号 ID。
	<code>TCR_PASSWORD</code>	开通 TCR 时设置的访问密码。
	<code>SSH_HOST</code>	服务器 IP。
	<code>SSH_USER</code>	如 <code>root</code> 。
	<code>SSH_KEY</code>	部署专用私钥全文 (非个人日常密钥)。

8.2 服务器 `/opt/blog` 目录

`compose.prod.yml`、`.env` (含镜像地址与全部密钥)、`config.yaml`、`nginx/*.conf` (`blog/admin/default` 三件 + `certs/` 证书目录)。首次部署的手工初始化步骤见 `docs/deployment-guide.md`。

九、已知限制与演进方向

- 编排与 `nginx conf` 不随 CI 分发: `deploy job` 只触发服务器重启重读, `conf` 文件本身的更新仍需手工 `scp` 到 `/opt/blog`——计划在 `deploy` 步骤前加 `scp` 同步。

- **无测试门禁**：server 目前没有单测（验证靠 build/vet），前端靠 tsc；CI 里尚未加测试 job，后端补测试后应在 build-push 前插入 test 门禁。
- **单实例部署**：无蓝绿/灰度，发版间隙（秒级）服务重启不可用；量起来后可考虑多实例 + 健康检查滚动。
- **HTTPS 未启用**：443 块已预写在 nginx conf（注释状态），等备案通过、证书放入 certs/ 后解注释并把 80 改为 301 跳转，同时删除 8081/8082 预览口。